

Simulación de Entrevista Técnica — SAGA & Outbox

Guía de estudio en formato entrevista · patrón SAGA por orquestación y patrón Transactional Outbox en el sistema POS

Guía de estudio en formato entrevista sobre el **patrón SAGA** (orquestado por `venta-service`) y el **patrón Outbox** para publicación fiable de eventos. Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y **follow-ups**.

Formato sugerido: 50–60 min. Bloques: SAGA (20'), compensación e idempotencia (15'), Outbox y dual-write (15'), escenarios (10').

0. Warm-up — SAGA en el proyecto

P Describe el flujo SAGA de una venta.

R `venta-service` es el **orquestador**:

1. `POST /api/ventas` crea la orden en **PENDING** y produce a `stock-reserve-topic`.
2. `stock-service` consume, valida disponibilidad, reserva y responde en `stock-reserve-response-topic`.
3. Si `success=true` → **STOCK_RESERVED** y produce a `despacho-request-topic`. Si no → **STOCK_FAILED** (terminal).
4. `despacho-service` genera tracking `TRK-XXXXXXX`, crea el despacho en **PROCESSING** y responde en `despacho-response-topic`.
5. Si `success=true` → **COMPLETED**. Si no → **DISPATCH_FAILED** y produce a `stock-compensate-topic` para liberar el stock (compensación).

Es una **SAGA por orquestación**: un coordinador central dirige pasos y compensaciones.

1. Patrón SAGA

P1 ¿Qué problema resuelve SAGA y por qué no una transacción distribuida ACID?

R En microservicios con **base-por-servicio** no hay una transacción ACID que abarque varias bases. SAGA descompone una transacción de negocio en una **secuencia de transacciones locales**, cada una con una **acción compensatoria** si algo falla más adelante. Se evita **2PC** (two-phase commit) porque bloquea recursos, acopla servicios, tiene un coordinador frágil y escala mal. SAGA acepta **consistencia eventual** a cambio de disponibilidad y desacoplamiento.

P2 ¿Orquestación vs coreografía? ¿Por qué orquestación aquí?

R **Orquestación**: un coordinador central (venta) invoca los pasos y decide las compensaciones → flujo **explícito, testeable y observable**. **Coreografía**: cada servicio reacciona a eventos sin coordinador → escala a muchos servicios pero **dispersa** la lógica y complica el rastreo. Con solo 3 pasos y una compensación no trivial, orquestación es la elección correcta.

P3 Modela la máquina de estados de la orden. ¿Cuáles son terminales?

R PENDING → STOCK_RESERVED → (DISPATCHED) → COMPLETED en el camino feliz. Terminales de fallo: **STOCK_FAILED** (sin stock) y **DISPATCH_FAILED** (falló despacho, ya compensado). **CANCELLED** para cancelación. Modelarlo como máquina de estados evita transiciones inválidas (regla: "solo órdenes con stock reservado pueden solicitar despacho").

2. Compensación e idempotencia

P4 ¿Cómo se implementa la compensación?

R Cada paso tiene una **acción compensatoria semántica** (no un rollback físico): la reserva de stock se compensa **liberando** la cantidad reservada vía `stock-compensate-topic`. Durante la SAGA el sistema puede estar temporalmente inconsistente (stock reservado, venta no completada); las compensaciones reconcilian el estado.

FOLLOW-UP ¿Y si la compensación falla? — Debe ser **idempotente y reintentable**; en producción se añade **DLQ** (dead-letter topic), reintentos con backoff y alertas para intervención manual como último recurso.

P5 Kafka entrega "at-least-once". ¿Qué riesgos y cómo los mitigas?

R Un consumidor puede procesar el mismo evento dos veces (p. ej. tras un rebalanceo), causando doble reserva o doble despacho. Mitigaciones:

- **Claves de idempotencia** por `orderId` .
- Operaciones **idempotentes** (upsert por `orderId`; verificar estado actual antes de aplicar).
- **Deduplicación** de eventos procesados (tabla/set de IDs vistos).
- Compensaciones seguras si se repiten.

FOLLOW-UP ¿Y el orden de los mensajes? — Kafka ordena **por partición**. Particionar por `orderId` garantiza el orden dentro de una misma orden, que es lo que importa aquí.

3. El problema dual-write

P6 ¿Qué es el problema del "dual write" en este flujo?

R Cuando `venta-service` debe hacer **dos escrituras** en sistemas distintos: (1) guardar la orden en MongoDB y (2) publicar el evento en Kafka. No hay transacción que abarque ambos. Si guarda en Mongo y **cae antes de publicar**, la orden existe pero el evento nunca sale → la SAGA se queda colgada. Si publica y falla al guardar, hay un evento sin orden. Los dos recursos no se pueden confirmar atómicamente.

P7 ¿Por qué no basta con "guardar y luego publicar"?

R Porque entre ambas operaciones hay una **ventana de fallo**. Tampoco sirve publicar primero (podría no guardarse). Cualquier orden de dos operaciones no atómicas deja un estado inconsistente posible. La solución es convertir las dos escrituras en **una sola escritura local transaccional** → patrón **Outbox**.

4. Patrón Transactional Outbox

P8 Explica el patrón Transactional Outbox.

R En la **misma transacción local** que guarda la orden, se inserta el evento en una tabla/colección **outbox**. Como ambas escrituras van a la misma base, son atómicas. Luego un proceso aparte (**relay/publisher**) lee la outbox y publica a Kafka, marcando cada evento como enviado. Así nunca hay orden sin evento ni evento sin orden.

```
@Transactional
public void crearVenta(Orden orden) {
    ordenRepo.save(orden);           // 1) estado de negocio
    outboxRepo.save(new OutboxEvent( // 2) evento, MISMA transacción
        orden.getId(), "stock-reserve", payload));
}
// Proceso relay (aparte): lee outbox PENDING → publica a Kafka → marca SENT
```

P9 ¿Cómo se publica desde la outbox? ¿Polling o CDC?

R Dos enfoques:

- **Polling publisher**: un scheduler consulta periódicamente los eventos **PENDING**, los publica y los marca **SENT**. Simple, funciona con Mongo o SQL.
- **CDC (Change Data Capture)**, p. ej. **Debezium**: lee el *oplog*/WAL de la base y publica los cambios a Kafka sin polling. Más eficiente y de baja latencia, pero añade infraestructura.

En ambos casos la publicación es **at-least-once**, así que los consumidores deben ser idempotentes.

FOLLOW-UP ¿Relación con Outbox e idempotencia? — La outbox garantiza que el evento **se publica** (no se pierde); la idempotencia del consumidor garantiza que **procesarlo varias veces** no causa daño. Se necesitan ambas: entrega fiable + procesamiento seguro.

P10 ¿El proyecto implementa Outbox hoy? ¿Cómo lo añadirías?

R El diseño actual publica directamente a Kafka tras guardar (dual-write, con la ventana de fallo descrita). Para endurecerlo añadiría una **colección outbox** en cada servicio productor (venta, stock, despacho), escritura del evento **dentro de la misma transacción** que el cambio de estado, y un **relay** (polling o Debezium) que publique y marque enviado. Esto convierte la publicación de eventos en fiable de extremo a extremo.

5. Diseño abierto / escenarios

P11 despacho-service cae 30 minutos. ¿Qué pasa con las ventas?

R Las órdenes quedan en **STOCK_RESERVED** con su evento retenido en despacho-request-topic (Kafka lo persiste). Al volver despacho, consume el backlog y la SAGA continúa. Riesgo: stock reservado retenido mientras tanto. Mitigaciones: **timeouts** en la SAGA que compensen si despacho no responde en X, y monitoreo del **lag** del consumer group.

P12 ¿Cómo observas una SAGA que cruza 3 servicios y Kafka?

R **Tracing distribuido** (OpenTelemetry) propagando un **correlationId** desde el gateway a través de las cabeceras de los eventos Kafka; **métricas** (duración de la SAGA, tasa de compensaciones, lag de Kafka) vía Micrometer → Prometheus; y **logs estructurados** correlacionados por **orderId**.

Apéndice — Chuleta rápida

Tema	Punto clave
SAGA	Transacciones locales + compensaciones; orquestada por venta-service
Por qué no 2PC	Bloquea recursos, acopla, coordinador frágil, escala mal
Compensación	Semántica (liberar stock vía <code>stock-compensate-topic</code>), idempotente
Estados	PENDING → STOCK_RESERVED → COMPLETED; fallos: STOCK_FAILED / DISPATCH_FAILED
at-least-once	Idempotencia por <code>orderId</code> ; orden por partición
Dual-write	Guardar en Mongo + publicar en Kafka no es atómico → ventana de fallo
Outbox	Evento en la MISMA transacción local; relay publica después (polling/CDC)
Relay	Polling publisher o Debezium (CDC del oplog/WAL); sigue siendo at-least-once
Observabilidad	<code>correlationId</code> en cabeceras Kafka + métricas de SAGA + logs por <code>orderId</code>